

I have two scripts for building a small, robust gene-expression classifier, but I want a concise explanation aimed at someone with a computer science background (not a biologist). Please explain the purpose and implementation of each script at a systems/ML level. Avoid biology details except for minimal definitions. Use plain language, brief equations only when essential, and include small ASCII diagrams or bullet flows if helpful.

Context you should assume:

Data setting: Two studies of oysters labeled by phenotype: condition $\in$ {tolerant, sensitive}. One large batch (A) and two smaller batches (B1, B2). Goal: a small, reproducible gene panel and a simple classifier that generalizes across batches.

Files:

Expression matrix from nf-core/differentialabundance, DESeq2 VST output: all.vst (1).tsv

- Shape: genes $\times$ samples.
- First column gene_id, remaining columns are sample IDs (e.g., SRX...).
- Values are continuous, already variance-stabilized.

Metadata CSV (original, uncleaned):

rnaseq_diffabundance_study1and5_samplesheet_filled.csv

- Columns: sample, condition, treatment, batch, Collection_Interval_Days.
- condition has text values tolerant or sensitive.
- batch distinguishes A, B1, B2 (or site/year strings that can be remapped to A/B1/B2).

Optional gene list: panel_candidates_tier12.txt (a curated subset of genes to prioritize).

Script 1 (R): Meta-analysis + Tiering

Explain what it does and why:

Purpose: Combine differential-expression evidence from Study A and B to rank genes by cross-study reproducibility, then assign each gene to Tier 1/2/3 based on significance, effect-direction agreement, and heterogeneity. Export a shortlist for modeling.

Inputs: Per-study DE results tables (for A, B, and combined C): columns like gene_id, log2FC, SE, p, padj, baseMean.

Core steps (tie them to standard CS ideas):

Define a common universe of features (intersection of gene IDs across A and B) to avoid training on features absent in one study.

Compute per-gene fixed-effect and random-effects meta-estimates; derive heterogeneity (I^2) and choose the meta p-value accordingly (FE if I^2 is low, RE if higher).

Enforce sign consistency (feature effect direction agrees across A and B).

Build a reproducibility score = $-\log_{10}(p_{\text{meta}}) - \lambda \cdot I^2 + \text{bonuses}$ (bonuses for being significant in A, B, and combined C, and for three-way sign agreement).

Tiering rules:

- Tier 1: replicated, consistent, low heterogeneity;
- Tier 2: supported but weaker;
- Tier 3: power-driven from the combined run only.

Outputs:

- meta_ranked.csv (per-gene meta stats and score),
- tiers.csv (tier assignments and flags),
- panel_candidates_tier12.txt (Tier1+2 genes for modeling).

What to emphasize for CS audience: It's feature screening

with cross-domain stability checks: meta-analysis $\approx$ weighted averaging of per-domain coefficients; $I^2 \approx$ between-domain variance; tiers $\approx$ priority queues for feature selection downstream.

Script 2 (Python): Classifier training with stability selection + LOSO

Explain what it does and why:

Purpose: Train a small, auditable classifier that generalizes across batches (domains). Use stability selection with L1 logistic regression to pick a minimal gene subset, then Leave-One-Group-Out (LOSO) validation by batch.

Inputs:

VST expression (all.vst (1).tsv) — transposed to samples $\times$ genes and aligned to metadata by sample.

Raw metadata CSV — cleaned in-memory (strip headers/whitespace, lowercase condition).

Optional panel_candidates_tier12.txt to restrict features up front.

Core steps (map to CS/ML concepts):

Data plumbing: transpose genes $\times$ samples to samples $\times$ genes; align rows to metadata; encode condition to {0,1}; use batch as the group label for LOSO.

Optional feature restriction: If a Tier1+Tier2 file is provided, slice columns to that set; otherwise use all genes.

Stability selection: Repeated, stratified subsampling by batch $\times$ class; for each resample, fit sparse logistic regression (L1/elastic net) and record which features have non-zero coefficients. Aggregate to get selection probabilities per gene (feature stability under resampling).

Redundancy pruning: Greedy removal of highly correlated features ($|r| > 0.9$) to avoid duplicates.

Panel-size sweep: Evaluate candidate panel sizes (e.g., 6, 8, 10, 12, 16, 24 genes). For each size, refit a simple logistic model (no penalty), apply LOSO by batch, and compute AUROC/AUPR/Brier. Choose the smallest panel within $\Delta\text{AUROC} \leq 0.02$ of the best.

Calibration: Use Platt scaling within the training fold to calibrate probabilities. Summarize performance by equal-weight averaging across held-out batches (prevents the large batch from dominating).

Outputs:

- `stability_selection.csv` (gene, selection probability),
- `panel_size_sweep_metrics.csv` (metrics vs panel size),
- `final_panel_gene_list.txt` (chosen minimal set),
- `loso_by_heldout.csv` (per-batch results).

What to emphasize for CS audience:

LOSO = domain generalization test (train on some domains, test on a held-out domain).

Stability selection = feature selection under resampling noise, akin to model-agnostic importance with sparsity.

The workflow separates feature discovery (R script, cross-domain reproducibility) from classifier construction (Python, domain-robust selection), reducing overfitting and improving interpretability.

Please structure your explanation with:

A 1–2 paragraph high-level overview of the two-script pipeline and its goal.

For each script: inputs, outputs, and a step-by-step

description with CS analogies (feature screening, weighted averaging, domain shift, sparsity, cross-validation).

An ASCII flow diagram of data moving from files → R tiering → Python modeling → reports.

A short glossary: VST, meta-analysis (FE/RE), I^2 , LOSO, stability selection, calibration.

Key risks and mitigations (data leakage, batch imbalance, overfitting; why LOSO and equal-weight averaging help).

How to adapt knobs: change panel sizes, adjust resamples, restrict to Tier1+Tier2 vs all genes, cap the large batch during training.

Runtime ballpark on a laptop and which parameters most affect it.

Keep the answer focused on software/ML concepts; minimal biology.

High-level overview (*drawing only on the context you provided now*)

You have a two-script pipeline that separates **feature discovery** from **classifier construction** to reduce overfitting across batches (domains).

- **Script 1 (R)** aggregates evidence from two studies to **rank genes by cross-study reproducibility** and places them into Tiers (1–3) using meta-analysis, sign agreement, and heterogeneity. Output = a prioritized shortlist.
- **Script 2 (Python)** builds a **small, auditable classifier** using stability selection with sparse logistic regression and evaluates **domain generalization** via **Leave-One-Group-Out (LOSO)** by batch (A, B1, B2). Output = final gene panel and calibrated predictions with per-batch metrics.

This mirrors a standard CS/ML pattern:

- R script ≈ **feature screening with cross-domain stability checks**.
- Python script ≈ **sparse model + stability selection + domain holdout validation**.

Script 1 (R): Meta-analysis + Tiering

Inputs

- Per-study DE result tables for A, B, and a combined run C: `gene_id`, `log2FC`, `SE`, `p`, `padj`, `baseMean`.
- (Universe) intersect gene IDs between A and B.

Outputs

- `meta_ranked.csv` : per-gene FE/RE meta-stats, I^2 , score.
- `tiers.csv` : Tier assignment + flags (sign consistency, per-study significance).
- `panel_candidates_tier12.txt` : Tier1 $\cup$ Tier2 gene list for downstream modeling.

What it does (systems/ML view)

Goal: rank features (genes) by **cross-study reproducibility** and **effect consistency**.

1. Common feature universe

- Use the **intersection** of gene IDs across studies A and B so downstream training never sees a feature missing in a domain.
- CS analogy: ensure the same **feature space** across domains to avoid train/test schema drift.

2. Per-gene meta-estimates

- **Fixed-effect (FE)** meta: weighted average of per-study effects

$$\hat{\beta}_{\text{FE}} = \frac{\sum_i w_i \hat{\beta}_i}{\sum_i w_i}, \quad w_i = \frac{1}{SE_i^2}$$

- **Random-effects (RE)** meta: FE plus an estimated between-study variance τ^2 (e.g., DerSimonian–Laird or REML), which down-weights disagreement between studies.

3. Heterogeneity & model choice

- Cochran's Q and I^2 quantify between-study variance:

$$I^2 = \max \left(0, \frac{Q - (k - 1)}{Q} \right) \times 100\%$$

- Choose FE if I^2 is low, otherwise RE.
- CS analogy: FE = assume **same effect** across domains; RE = allow **domain-specific shifts**.

4. Sign consistency

- Require **effect direction agreement** (e.g., log2FC signs) across A and B.

- CS analogy: keep features with **stable coefficient sign** across domains.

5. Reproducibility score

- Core idea:

$$\text{score} = -\log_{10}(p_{\text{meta}}) - \lambda \cdot I^2 + \text{bonuses}$$

Bonuses for: per-study significance (A, B), combined-run significance (C), and **three-way sign agreement** (A, B, C).

- CS analogy: **ranking function** combining strength of evidence and cross-domain stability, penalizing heterogeneity.

6. Tiering

- **Tier 1**: replicated, consistent sign, **low** heterogeneity (stable across A & B).
- **Tier 2**: supported but weaker (e.g., smaller effect or moderate I^2).
- **Tier 3**: primarily supported by **combined** run C (power-driven, weaker cross-domain evidence).
- CS analogy: Tiers = **priority queues** feeding downstream sparse selection.

Why this helps

You screen features using **cross-domain statistics** before any classifier sees them. That reduces the search space and biases selection toward **portable** signals that survive domain shift (A vs B1/B2).

Script 2 (Python): Classifier training with stability selection + LOSO

Inputs

- `all.vst (1).tsv` : VST expression (genes × samples).
- `rnaseq_diffabundance_study1and5_samplesheet_filled.csv` : raw metadata (sample, condition, batch, etc.).
- Optional: `panel_candidates_tier12.txt` (from R).

Outputs

- `stability_selection.csv` : genes with selection probabilities.
- `panel_size_sweep_metrics.csv` : AUROC/AUPR/Brier vs panel size.
- `final_panel_gene_list.txt` : chosen minimal set.
- `loso_by_heldout.csv` : metrics per held-out batch (A, B1, B2).

What it does (systems/ML view)

Goal: produce a **small, interpretable** classifier that **generalizes across batches**.

1. Data plumbing

- Transpose expression to **samples × genes**; align rows to metadata via sample IDs.
- Normalize labels: $\text{condition} \in \{\text{tolerant}, \text{sensitive}\} \rightarrow \{1, 0\}$.
- Use $\text{batch} \in \{A, B1, B2\}$ as **group labels** for LOSO.

2. Optional feature restriction

- If `panel_candidates_tier12.txt` exists, **slice columns** to that set; else use all genes.
- CS analogy: **prior** feature filter from Script 1.

3. Stability selection (sparse feature discovery)

- Repeat **stratified subsampling** that respects **batch × class** balance.
- Fit **L1 (or elastic net) logistic regression** on each resample:

$$\min_{\beta} \text{logloss}(X\beta, y) + \alpha \|\beta\|_1$$

- Record features with **nonzero** coefficients; aggregate across resamples → **selection probability** per gene.
- CS analogy: measure **feature stability under resampling noise**; akin to model-agnostic importance with a sparsity prior.

4. Redundancy pruning

- Greedy removal of features with $|r| > 0.9$ (highly correlated), preserving the more stable one.
- CS analogy: reduce **multicollinearity** / duplicate features.

5. Panel-size sweep

- Sort features by stability; evaluate sizes (e.g., 6, 8, 10, 12, 16, 24).
- For each size: refit a **plain** logistic model (no penalty) on selected features, run **LOSO by batch**, compute **AUROC, AUPR, Brier**.
- Choose the **smallest** panel within $\Delta\text{AUROC} \leq 0.02$ of the best.
- CS analogy: **capacity control** with a **performance plateau** criterion.

6. Calibration

- **Platt scaling** within each training fold; evaluate on the held-out batch.
- Aggregate metrics by **equal-weight averaging across batches** (prevents big batch A from dominating scores).

Why this helps

- LOSO is a **domain generalization** test: train on two batches, test on the third.

- **Stability selection + redundancy pruning** drives a **minimal** panel that is **robust** under resampling and less likely to overfit domain-specific quirks.

ASCII flow

Files



```
|
|— all.vst (1).tsv      → (genes × samples; VST values)
|— rnaseq_diffabundance_study1and5_samplesheet_filled.csv (metadata)
|— (optional) per-study DE tables: A, B, combined C
  ↓
[Script 1: R] Meta-analysis + Tiering
- Intersect features (A ∩ B)
- FE/RE meta, I2, sign checks
- Score & Tiers (1/2/3)
- Outputs: meta_ranked.csv, tiers.csv, panel_candidates_tier12.txt
  ↓ (Tier1UTier2 shortlist)
[Script 2: Python] Classifier + Validation
- Load VST, align to metadata
- (Optional) restrict to Tier1UTier2
- Stability selection (L1 LR) + pruning
- Panel-size sweep + LOSO by batch
- Platt calibration, equal-weight batch averaging
- Outputs: stability_selection.csv, final_panel_gene_list.txt,
          panel_size_sweep_metrics.csv, loso_by_heldout.csv
```

Glossary (short)

- **VST**: Variance-Stabilizing Transform; makes gene-wise variance ~constant so linear models behave better.
- **Meta-analysis (FE/RE)**: Combine study-level effects: **FE** assumes one true effect; **RE** allows per-study variability.
- **I²**: % of total variation due to **between-study heterogeneity** (higher = more disagreement).
- **LOSO (Leave-One-Group-Out)**: Cross-validation where each **batch** (A, B1, B2) is held out in turn; tests domain generalization.
- **Stability selection**: Repeated subsampling + sparse model; select features that recur often → **selection probability**.

- **Calibration (Platt):** Map raw scores → calibrated probabilities via a logistic link fitted on the training fold.
-

Key risks and mitigations

- **Data leakage**
 - *Risk:* Using all data to choose features, then evaluating on a held-out batch.
 - *Mitigation:* Run **stability selection + panel-size sweep entirely within each LOSO training fold**. Treat the R shortlist as a **prior filter** only; avoid reusing any per-batch test information inside a fold.
 - **Batch imbalance ($A \gg B1, B2$)**
 - *Risk:* Metrics dominated by the large batch; model overfits A.
 - *Mitigation:* **Equal-weight averaging across held-out batches**; optionally **cap** samples per batch during training or use class/batch-balanced resampling.
 - **Overfitting via correlated features**
 - *Risk:* Many near-duplicates inflate confidence.
 - *Mitigation:* **Redundancy pruning** ($|r| > 0.9$) after stability selection.
 - **Threshold/knob sensitivity**
 - *Risk:* Tier cutoffs (l^2 , p_{meta}), λ in the score, correlation threshold, panel-size tolerance Δ may shift the panel.
 - *Mitigation:* Keep defaults conservative; record a **config** for reproducibility; run **sensitivity checks**.
 - **Using combined run C**
 - *Risk:* If combined C influences selection and also appears in test folds, it can bias tiers.
 - *Mitigation:* Use C only for **bonus flags**, not to override A/B disagreement; keep **sign consistency** primary.
-

Adaptable knobs

- **R (Tiering)**
 - l^2 thresholds for FE vs RE; λ in score; p-value cutoffs for Tier 1/2; whether to require strict sign agreement.
- **Python (Selection & eval)**
 - Resamples (e.g., 200–1000), subsample fraction (e.g., 0.6–0.8), L1/EN penalty grid.
 - Correlation pruning threshold (e.g., $|r| > 0.9$).

- Panel sizes to sweep and ΔAUROC tolerance (e.g., 0.02).
- Whether to **restrict to Tier1+2** vs use all genes.
- Batch **capping** during training to reduce dominance of A.

Runtime ballpark (recent laptop)

- **Script 1 (R)**: seconds to a few minutes (meta across tens of thousands of genes is cheap).
 - **Script 2 (Python)**: dominated by **stability selection** (many sparse LR fits).
 - If restricted to Tier1+2 (hundreds of genes), **~5–20 min** for 500 resamples and a small panel sweep.
 - Using all genes (10k–20k) or >1000 resamples can push to **30–90+ min**.
 - Most expensive knobs: **# resamples**, **feature count** entering L1, and **panel-size grid**.
-

Minimal equations (for reference)

- FE meta: $\hat{\beta}_{\text{FE}} = \frac{\sum_i \hat{\beta}_i / SE_i^2}{\sum_i 1 / SE_i^2}$
- I^2 : $I^2 = \max\left(0, \frac{Q - (k-1)}{Q}\right) \times 100\%$
- L1 logistic: $\min_{\beta} \text{logloss}(X\beta, y) + \alpha \|\beta\|_1$

☰ ☞ > ChatGPT ▾

✍️ ⬆️ ⋮

Considerations you might not have thought of

- **Nested feature selection per LOSO fold**: If you want *strict* no-leakage, re-run the stability selection **inside each LOSO training fold**, not once globally.
- **Model card & audit trail**: Save seeds, config, package versions, and hashes of input files; write out selected features with their **selection probabilities** and **pairwise correlations**.
- **Calibration diagnostics**: Report ECE or reliability curves per held-out batch in addition to Brier.
- **Thresholding for decisions**: If you plan to act on probabilities, pick thresholds per batch using the **training folds only**.
- **Alternative baselines**: Compare to a **Linear SVM** on the same selected features; sometimes margins are more stable under domain shift.

- **Class priors:** If class balance differs by batch, consider **prior correction** in the held-out evaluation to avoid optimistic/biased probabilities.
-

Hallucination risk: 3/10 (low–moderate).

Reasoning: The description is directly based on your provided setup. Small risk remains in runtime ranges and specific implementation details (e.g., exact RE estimator, exact subsampling rates) that can vary by codebase defaults.

